

ML, Time Series & Forecasting — Complete Reference

EY Probation Training — Week 2 (ML intro, time series, feature engineering, forecasting models, forecast evaluation) and Week 3 (FEU forecasting KPIs). Dataset: Kaggle Superstore.

The whole roadmap is **forecasting-shaped**. This is not general ML — it is regression-based demand forecasting with supply-chain KPIs. Frame everything through that lens.

1. Supervised Learning Basics

Supervised = labelled data. Regression predicts a continuous value; classification predicts a category. Forecasting is regression with a time dimension and ordering constraints.

Unsupervised = no labels (clustering, PCA, anomaly detection).

Bias-variance tradeoff

- **High bias / underfitting** — model too simple, poor on both train and test. Fix: more features, more capacity, less regularisation.
- **High variance / overfitting** — model memorises noise, great on train and poor on test. Fix: more data, fewer features, more regularisation, simpler model, early stopping.

Diagnose by comparing train and validation error. Small gap and both bad = bias. Large gap = variance.

The core workflow

1. Frame the problem and choose the metric **before** modelling.
2. Split the data (chronologically, for time series).
3. Explore and clean.
4. Engineer features.
5. Fit a **baseline first**.
6. Train candidate models.
7. Evaluate on held-out data with the chosen metric.
8. Interpret, then iterate.

Never skip step 5. A model that can't beat naive-forecast or seasonal-naive isn't worth deploying, and the baseline comparison is the first thing a reviewer asks for.

2. Regression Models

Linear regression

Fits $y = b_0 + b_1x_1 + \dots + b_nx_n$ by minimising squared error. Fast, interpretable coefficients, but assumes linearity and additivity, and is sensitive to outliers and multicollinearity.

Regularised variants: - **Ridge (L2)** — shrinks coefficients toward zero, handles multicollinearity, never zeroes them out. - **Lasso (L1)** — can drive coefficients to exactly zero, so it performs feature selection. - **ElasticNet** — a mix of both.

Tree-based models

Decision tree — recursive splits minimising impurity (MSE for regression). Interpretable but high variance; prone to overfitting unless depth-limited.

Random Forest — many trees on **bootstrap samples**, each split considering a **random subset of features**, predictions averaged. Bagging: reduces variance through averaging. Trees are built independently and in parallel.

Gradient Boosting / XGBoost — trees built **sequentially**, each fitting the residual errors of the ensemble so far. Reduces bias. More accurate than RF on tabular data but more sensitive to hyperparameters and easier to overfit.

	Random Forest	XGBoost
Trees built	Independently, in parallel	Sequentially, on residuals
Reduces	Variance (bagging)	Bias (boosting)
Overfitting risk	Low	Higher — needs tuning and early stopping
Tuning sensitivity	Forgiving	Sensitive
Handles NaN natively	No (sklearn)	Yes
Typical use	Strong quick baseline	Squeezing out accuracy

Key hyperparameters:

```
RandomForestRegressor(n_estimators=300, max_depth=None, min_samples_leaf=5,
                      max_features="sqrt", n_jobs=-1, random_state=42)

XGBRegressor(n_estimators=1000, learning_rate=0.05, max_depth=6,
             subsample=0.8, colsample_bytree=0.8,
             reg_alpha=0, reg_lambda=1, early_stopping_rounds=50,
             random_state=42)
```

Lower `learning_rate` needs more `n_estimators`. `subsample` and `colsample_bytree` add randomness that fights overfitting. Use `early_stopping_rounds` with a validation set rather than guessing `n_estimators`.

Feature importance: tree-based `.feature_importances_` is impurity-based and biased toward high-cardinality features. **Permutation importance** and **SHAP** are more trustworthy. Importance is not causation.

3. Time Series Fundamentals

Components

- **Trend** — long-run direction.
- **Seasonality** — a repeating pattern of **fixed, known period** (weekly, monthly, yearly).
- **Cyclic** — repeating but of **variable, unknown length** (business cycles). Not the same as seasonality; a common exam distinction.
- **Residual / irregular** — what's left.

Decomposition: - **Additive:** $y = \text{Trend} + \text{Seasonality} + \text{Residual}$ — when seasonal swings are roughly constant in absolute size. -

Multiplicative: $y = \text{Trend} \times \text{Seasonality} \times \text{Residual}$ — when swings grow with the level. A log transform converts multiplicative to additive.

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL
seasonal_decompose(series, model="additive", period=12).plot()
STL(series, period=12).fit().plot()      # more robust to outliers
```

Stationarity

A series is stationary if mean, variance and autocovariance don't change over time. ARIMA-family models require it.

Testing: - **ADF (Augmented Dickey-Fuller):** H_0 = has a unit root = **non-stationary**. $p < 0.05 \rightarrow$ reject $H_0 \rightarrow$ stationary. - **KPSS:** H_0 = **stationary** — the opposite null. $p < 0.05 \rightarrow$ non-stationary.

Getting the null hypotheses the right way round is a classic exam trap. Running both and agreeing is best practice.

Making a series stationary: differencing ($y_t - y_{t-1}$), seasonal differencing ($y_t - y_{t-s}$), log or Box-Cox transform for non-constant variance, or explicit de-trending.

ACF and PACF

- **ACF** — correlation of the series with its own lags, including indirect effects. Slow decay indicates a trend (non-stationary). Spikes at multiples of s indicate seasonality.
- **PACF** — correlation at lag k with the effects of intermediate lags removed.

Pattern	Suggests
ACF cuts off after lag q , PACF tails off	MA(q)
PACF cuts off after lag p , ACF tails off	AR(p)
Both tail off	ARMA(p, q)
ACF decays very slowly	Non-stationary — difference it

4. Univariate Models

ARIMA(p, d, q) — p autoregressive terms, d differences, q moving-average terms. **SARIMA(p, d, q)(P, D, Q) s** — adds a seasonal component with period s .

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
m = SARIMAX(y, order=(1,1,1), seasonal_order=(1,1,1,12)).fit()
fc = m.get_forecast(steps=12); fc.predicted_mean; fc.conf_int()
```

Exponential smoothing (Holt-Winters) — weighted averages with exponentially decaying weights. - Simple ES: level only, no trend or seasonality. - Holt's linear: level + trend. - Holt-Winters: level + trend + seasonality (additive or multiplicative).

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
ExponentialSmoothing(y, trend="add", seasonal="add", seasonal_periods=12).fit()
```

Baselines you must always compute: - **Naive:** forecast = last observed value. - **Seasonal naive:** forecast = value from the same period last season. Usually the benchmark to beat in demand forecasting. - **Moving average / drift.**

5. Feature Engineering for Forecasting

This is the heart of Week 2 — ML-based forecasting turns a time series into a supervised tabular problem.

Lag features

```
for L in [1, 2, 3, 7, 14, 28]:
    df[f"lag_{L}"] = df.groupby("sku")["sales"].shift(L)
```

Choose lags by horizon. If you must forecast 7 days ahead, `lag 1` will not exist at prediction time — you'd be using data you won't have. Lags must be at least as large as the forecast horizon, or you need a recursive strategy.

Rolling / moving-average features

```
g = df.groupby("sku")["sales"]
df["roll_mean_7"] = g.transform(lambda s: s.shift(1).rolling(7).mean())
df["roll_std_7"] = g.transform(lambda s: s.shift(1).rolling(7).std())
df["roll_max_28"] = g.transform(lambda s: s.shift(1).rolling(28).max())
df["ewm_7"] = g.transform(lambda s: s.shift(1).ewm(span=7).mean())
```

The `.shift(1)` before `.rolling()` is mandatory. Without it the window includes the current row — the value you're trying to predict — and that is leakage. Your validation score will look excellent and the model will fail in production. This is the single most common forecasting bug.

Date-based features

```
d = df["date"].dt
df["year"], df["month"], df["day"] = d.year, d.month, d.day
df["dayofweek"], df["weekofyear"], df["quarter"] = d.dayofweek, d.isocalendar().week, d.quarter
df["is_weekend"] = d.dayofweek >= 5
df["is_month_start"] = d.is_month_start
df["days_since_start"] = (df.date - df.date.min()).dt.days # linear trend proxy
# cyclical encoding - so December is adjacent to January
df["month_sin"] = np.sin(2*np.pi*d.month/12)
df["month_cos"] = np.cos(2*np.pi*d.month/12)
```

Cyclical encoding matters: as a plain integer, month 12 and month 1 look eleven apart when they're actually adjacent.

Other useful features

Holiday and promotion flags; price and discount; days since last promotion; expanding mean; group-level aggregates (category mean, store mean); and **target encoding of categoricals — computed on the training fold only.**

6. Train/Test Splitting for Time Series

Never use a random split or plain K-fold. A random split lets the model see t+1 while predicting t. Split **chronologically**.

```
cutoff = df.date.max() - pd.Timedelta(days=90)
train, test = df[df.date <= cutoff], df[df.date > cutoff]
```

Walk-forward / rolling-origin validation:

```
from sklearn.model_selection import TimeSeriesSplit
for tr_idx, te_idx in TimeSeriesSplit(n_splits=5).split(X):
    ...
```

Each fold trains on everything up to a point and tests on the next block. Expanding window keeps all history; sliding window keeps a fixed recent span.

Leakage checklist

- Rolling/lag features must use `.shift()` so they exclude the current value.
- Scalers, imputers and encoders are **fit on train only**, then applied to test.
- No feature may use information unavailable at prediction time (e.g. end-of-month totals when predicting mid-month).
- With group structure (per SKU, per store), compute lags **within group** — a `shift` without `groupby` bleeds one SKU's history into another's.
- The test period must sit strictly after the train period, with a gap if the horizon requires it.

7. Forecast Evaluation

General regression metrics

Metric	Formula	Character
MAE	$\text{mean}(y - \hat{y})$	Same units as y; robust to outliers
MSE	$\text{mean}((y - \hat{y})^2)$	Penalises large errors heavily; squared units
RMSE	$\sqrt{\text{MSE}}$	Same units as y; outlier-sensitive
MAPE	$\text{mean}(y - \hat{y} / y) \times 100$	Scale-free %; undefined at y = 0 , asymmetric
sMAPE	symmetric variant	Handles zeros better
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Variance explained; can be negative

MAE vs RMSE: RMSE > MAE always (equal only when all errors are identical). A large gap between them indicates a few big errors. Choose MAE when all errors cost proportionally; RMSE when large misses are disproportionately costly (usually true in inventory).

MAPE's problems are worth knowing: it explodes near zero demand, and it penalises over-forecasting more than under-forecasting, which biases model selection toward under-forecasting. In intermittent demand it's often unusable.

Supply-chain forecasting KPIs

These appear in both Week 2 and Week 3 — a strong signal they'll be assessed.

Forecast Accuracy (FA) — commonly $\text{FA} = (1 - \text{MAPE}) \times 100\%$, or the WMAPE-based $\text{FA} = (1 - \Sigma|A-F| / \Sigma A) \times 100\%$. The weighted form is preferred because it doesn't blow up on low-volume items and it weights by actual volume.

Forecast Bias (FB) — **signed**, unlike the error metrics:

$$\text{FB} = \Sigma(F - A) / \Sigma A \times 100\%$$

- Positive → systematically **over**-forecasting → excess inventory, obsolescence.
- Negative → systematically **under**-forecasting → stockouts, lost sales.
- Near zero → unbiased, though it can still have large offsetting errors.

Bias is the one that matters most operationally. A forecast can have excellent accuracy and terrible bias, or acceptable accuracy with zero bias. Bias compounds over time in inventory; random error largely cancels. Always report accuracy and bias **together** — reporting only accuracy is the mistake.

Tracking Signal = cumulative bias / MAD. Sustained values outside roughly ± 4 indicate the model needs re-fitting.

WMAPE / weighted MAPE = $\Sigma|A-F| / \Sigma A$. The standard workhorse for a portfolio of SKUs.

Get FEU's own formulas from their documentation. Different organisations define FA and FB with different denominators (actuals vs forecast), sign conventions, and aggregation levels. Confidently giving the textbook version when theirs differs scores worse than asking.

Aggregation matters

Accuracy computed at SKU-store-day level is much worse than the same forecast measured at category-month level, because errors cancel on aggregation. Always state the level. Comparing two forecasts measured at different levels is meaningless.

8. Worked Pipeline (Superstore)

```

import pandas as pd, numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from xgboost import XGBRegressor

df = pd.read_csv("superstore.csv", parse_dates=["Order Date"], encoding="latin-1")

# 1. aggregate to a regular grid
ts = (df.groupby([pd.Grouper(key="Order Date", freq="W"), "Category"])["Sales"]
      .sum().reset_index().rename(columns={"Order Date": "date", "Sales": "sales"}))

# 2. complete the spine so gaps are explicit
spine = (pd.MultiIndex.from_product(
    [pd.date_range(ts.date.min(), ts.date.max(), freq="W"), ts.Category.unique()],
    names=["date", "Category"]).to_frame(index=False))
ts = spine.merge(ts, on=["date", "Category"], how="left").fillna({"sales": 0})
ts = ts.sort_values(["Category", "date"])

# 3. features – note shift(1) before every rolling window
g = ts.groupby("Category")["sales"]
for L in [1, 2, 4, 8, 52]:
    ts[f"lag_{L}"] = g.shift(L)
for W in [4, 8]:
    ts[f"roll_mean_{W}"] = g.transform(lambda s: s.shift(1).rolling(W).mean())
    ts[f"roll_std_{W}"] = g.transform(lambda s: s.shift(1).rolling(W).std())
ts["month"] = ts.date.dt.month
ts["week"] = ts.date.dt.isocalendar().week.astype(int)
ts["msin"], ts["mcos"] = np.sin(2*np.pi*ts.month/12), np.cos(2*np.pi*ts.month/12)
ts = ts.dropna()

# 4. chronological split
cut = ts.date.max() - pd.Timedelta(weeks=13)
train, test = ts[ts.date <= cut], ts[ts.date > cut]
feats = [c for c in ts.columns if c not in ("date", "Category", "sales")]
Xtr, ytr, Xte, yte = train[feats], train.sales, test[feats], test.sales

# 5. baselines FIRST
naive = test.groupby("Category")["lag_1"]
print("naive MAE:", mean_absolute_error(yte, test["lag_1"]))
print("seasonal naive MAE:", mean_absolute_error(yte, test["lag_52"]))

# 6. models
rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3,
                           random_state=42, n_jobs=-1).fit(Xtr, ytr)
xgb = XGBRegressor(n_estimators=600, learning_rate=0.05, max_depth=5,
                   subsample=0.8, colsample_bytree=0.8, random_state=42).fit(Xtr, ytr)

# 7. evaluate – accuracy AND bias together
def report(name, y, p):
    mae = mean_absolute_error(y, p)
    rmse = np.sqrt(mean_squared_error(y, p))
    wmape = np.abs(y - p).sum() / y.sum() * 100
    bias = (p - y).sum() / y.sum() * 100
    print(f"{name:8s} MAE {mae:9.1f} RMSE {rmse:9.1f} "
          f"WMAPE {wmape:6.2f}% FA {100-wmape:6.2f}% Bias {bias:+6.2f}%")

report("RF", yte, rf.predict(Xte))
report("XGB", yte, xgb.predict(Xte))

```

Report FA and bias side by side, and always against the baselines.

9. Practice Checklist

- ☐ Decompose a series; state additive vs multiplicative and why
- ☐ Run ADF and KPSS; state both null hypotheses correctly
- ☐ Read ACF/PACF plots and propose ARIMA orders
- ☐ Build lag, rolling and date features **with** the `.shift(1)` guard
- ☐ Explain three concrete leakage risks in a forecasting pipeline
- ☐ Chronological split plus `TimeSeriesSplit` walk-forward
- ☐ Fit naive and seasonal-naive baselines before any model
- ☐ Fit RF and XGBoost; explain bagging vs boosting
- ☐ Compute MAE, RMSE, MAPE, WMAPE, FA and FB on the same forecast

- ☐ Explain why bias matters more than accuracy operationally
- ☐ Show how the same forecast scores differently at two aggregation levels